Algorithm Design and Analysis

Fernando I. Mondragón Vega

October 5, 2026

Chapter 1

Some Representative Problems [1]

1.1 Solved Exercises

Exercise 1

Consider a town with n men and n women seeking to get married to one another. Each man has a preference list that ranks all the women, and each woman has a preference list that ranks all the men. The set of all 2n people is divided into two categories: good people and bad people. Suppose that for some number k, 1= k = n - 1, there are k good men and k good women; thus there are n - k bad men and n - k bad women. Everyone would rather marry any good person than any bad person. Formally, each preference list has the property that it ranks each good person of the opposite gender higher than each bad person of the opposite gender: its first k entries are the good people (of the opposite gender) in some order, and its next n - k are the bad people (of the opposite gender) in some order. Show that in every stable matching, every good man is married to a good woman.

Solution (By contradiction)
Let's suppose we get a stable matching **S** in which a certain good man **m** is married to a bad woman **w'**. That implies there's a certain good woman **w** that is married to a bad man **m'**, since there is the same amount of good men as good women (k). That means we have an instability, since **m** prefers **w** over **w'**, and **w** prefers **m** over **m'**. This scenario can only happen in two cases:

1. **m** never proposed to **w**. However, this cannnot happen, since **w** is ranked higher than **w'** in **m**'s preference list.

2. **w** rejected **m**. Once again, this is not possible, since **m** is ranked higher than **m'** in **w**'s preference list.

We have found a contradiction, so **m, w'** could have never match to begin with.

1.2 Exercises

1. There are many other settings in which we can ask questions related to some type of "stability" principle. Here's one, involving competition between two enterprises. Suppose we have two television networks, whom we'll call A and B. There are n prime-time programming slots, and each network has n TV shows. Each network wants to devise a schedule -an assignment of each show to a distinct slot- so as to attract as much market share as possible. Here is the way we determine how well the two networks perform relative to each other, given their schedules. Each show has a fixed rating, which is based on the number of people who watched it last year; we'll assume that no two shows have exactly the same rating. A network wins a given time slot if the show that it schedules for the time slot has a larger rating than the show the other network schedules for that time slot. The goal of each network is to win as many time slots as possible. Suppose in the opening week of the fall season, Network A reveals a schedule S and Network B reveals a schedule T. On the basis of this pair of schedules, each network wins certain time slots, according to the rule above. We'll say that the pair of schedules (S, T) is stable if neither network can unilaterally change its own schedule and win more time slots. That is, there is no schedule S' such that Network A wins more slots with the pair (S', T) than it did with the pair (S, T); and symmetrically, there is no schedule T' such that Network B wins more slots with the pair (S, T') than it did with the pair (S, T). The analogue of Gale and Shapleys question for this kind of stability is the following: For every set of TV shows and ratings, is there always a stable pair of schedules? Resolve this question by doing one of the following two things:

   (a) give an algorithm that, for any set of TV shows and associated ratings, produces a stable pair of schedules; or

   (b) give an example of a set of TV shows and associated ratings for which there is no stable pair of schedules.

   Solution (Disproof by counterexample)
   Let $R_A = \{1, 4\}$ and $R_B = \{2, 5\}$ be the ratings of TV shows in network A and B, respectively. We have 4 possible scenarios after each network reveals their schedules, as follows:

S	T
1	2
4	5

(a)

S	T
1	5
4	2

(b)

S	T
4	2
1	5

(c)

S	T
4	5
1	2

(d)

   (a) In this case, network A wins both spots, but network B can exchange its 2 programs so that it wins the first spot.

(b) In this case, both networks win one spot each. However network A can swap its shows so that it wins both spots.

(c) Network A can do the same as in case b)

(d) Network B can do the same as in case a)

Therefore, there is no stable pair of schedules in this example. □

2. Gale and Shapley published their paper on the Stable Matching Problem in 1962; but a version of their algorithm had already been in use for ten years by the National Resident Matching Program, for the problem of assigning medical residents to hospitals. Basically, the situation was the following. There were m hospitals, each with a certain number of available positions for hiring residents. There were n medical students graduating in a given year, each interested in joining one of the hospitals. Each hospital had a ranking of the students in order of preference, and each student had a ranking of the hospitals in order of preference. We will assume that there were more students graduating than there were slots available in the m hospitals.
The interest, naturally, was in finding a way of assigning each student to at most one hospital, in such a way that all available positions in all hospitals were filled. (Since we are assuming a surplus of students, there would be some students who do not get assigned to any hospital.) We say that an assignment of students to hospitals is stable if neither of the following situations arises.

- First type of instability: There are students s and s', and a hospital h, so that
  - s is assigned to h, and
  - s' is assigned to no hospital, and
  - h prefers s' to s.
- Second type of instability: There are students s and s', and hospitals h and h', so that
  - s is assigned to h, and
  - s' is assigned to h', and
  - h prefers s' to s, and
  - s' prefers h to h'.

So we basically have the Stable Matching Problem, except that (i) hospitals generally want more than one resident, and (ii) there is a surplus of medical students. Show that there is always a stable assignment of students to hospitals, and give an algorithm to find one.

Solution
Let's propose the following algorithm to assign students to hospitals.

Initially every position is open and every student is unassigned
while there is a hospital with an open position and it hasn't accepted every student **do**

Choose such a hospital h
Let s be the highest ranked student in h's preference list to whom h has not yet accepted
if s is not yet assigned **then**
(h, s) ▷ s gets assigned to h
else if s is already assigned to h' **then**
if s prefers h to h' **then**
(h, s) ▷ s gets assigned to h
A position gets available in h'
else if s prefers h' to h **then**
h still has an open position
end if
end if
end while
return the set S of assignments

Observation 1 Hospitals accept students in decreasing order of preference.
Observation 2 Once a student is accepted in a hospital, they never become unassigned, they only accept better offers.
Now, let's prove the set of assignments S is stable.
Claim There is no unstable assignment.
Proof (By contradiction)

- First type of instability
  Suppose (h, s') is an unstable assignment of the first type: h prefers s' to s but s' is not assigned to any hospital and s is assigned to h.
  $\rightarrow$ h never accepted s', since s' is unassigned.
  $\rightarrow$ h prefers its current students (including s) to s'.
  $\rightarrow$ There's no instability.

  But that's a contradiction!
- Second type of instability
  Suppose (h, s') is an unstable assignment of the second type: h prefers s' to s and h' prefers s to s' but s is assigned to h and s' is assigned to h'.
  Case 1. h never accepted s'
  $\rightarrow$ h prefers its current students to s'.
  $\rightarrow$ There's no instability.

  Case 2. h accepted s'
  $\rightarrow$ s' refused (right away o later).
  $\rightarrow$ s' prefers h' over h.
  $\rightarrow$ There's no instability.

  In either case there's no instability, a contradiction!

3. The Stable Matching Problem, as discussed in the text, assumes that all men and women have a fully ordered list of preferences. In this problem we will consider a version of the problem in which men and women can be indifferent between certain options. As before we have a set M of n men and a set W of n women. Assume each man and each woman ranks the members of the opposite gender, but now we allow ties in the ranking. For example (with n = 4), a woman could say that m1 is ranked in first place; second place is a tie between m2 and m3 (she has no preference between them); and m4 is in last place. We will say that w prefers m to m' if m is ranked higher than m' on her preference list (they are not tied). With indifferences in the rankings, there could be two natural notions for stability. And for each, we can ask about the existence of stable matchings, as follows.

   (a) A strong instability in a perfect matching S consists of a man m and a woman w, such that each of m and w prefers the other to their partner in S. Does there always exist a perfect matching with no strong instability? Either give an example of a set of men and women with preference lists for which every perfect matching has a strong instability; or give an algorithm that is guaranteed to find a perfect matching with no strong instability.

   (b) A weak instability in a perfect matching S consists of a man m and a woman w, such that their partners in S are w' and m', respectively, and one of the following holds:

   - m prefers w to w', and w either prefers m to m' or is indifferent between these two choices; or
   - w prefers m to m', and m either prefers w to w' or is indifferent between these two choices.

   In other words, the pairing between m and w is either preferred by both, or preferred by one while the other is indifferent. Does there always exist a perfect matching with no weak instability? Either give an example of a set of men and women with preference lists for which every perfect matching has a weak instability; or give an algorithm that is guaranteed to find a perfect matching with no weak instability.

Solution (By counterexample)
Let the following be the preference lists of 2 men and 2 women.

	1st	2nd
Yancey	Anne, Becky	
Zeus	Becky	Anne

	1st	2nd
Anne	Yancey	Zeus
Becky	Yancey	Zeus

After a slight modification to Gale-Shapley algorithm in which a man proposes to one of its tied options at random, we can get the next matchings:

- $(Y, A), (Z, B)$. In which Becky would prefer being matched with Yancey but Yancey is indifferent between her and Anne (his current match). This is a weak instability.
- $(Y, B), (Z, A)$. In which Anne would prefer being matched with Yancey but Yancey is indifferent between her and Becky (his current match). This is also a weak instability.

Therefore, every matching produced by G-S has a weak instability in this case. □

4. Some of your friends are working for CluNet, a builder of large communication networks, and they are looking at algorithms for switching in a particular type of input/output crossbar. Here is the setup. There are n input wires and n output wires, each directed from a source to a terminus. Each input wire meets each output wire in exactly one distinct point, at a special piece of hardware called a junction box. Points on the wire are naturally ordered in the direction from source to terminus; for two distinct points x and y on the same wire, we say that x is upstream from y if x is closer to the source than y, and otherwise we say x is downstream from y. The order in which one input wire meets the output wires is not necessarily the same as the order in which another input wire meets the output wires. (And similarly for the orders in which output wires meet input wires.) Figure 1.8 gives an example of such a collection of input and output wires. Now, here' s the switching component of this situation. Each input wire is carrying a distinct data stream, and this data stream must be switched onto one of the output wires. If the stream of Input i is switched onto Output j, at junction box B, then this stream passes through all junction boxes upstream from B on Input i, then through B, then through all junction boxes downstream from B on Output j. It does not matter which input data stream gets switched onto which output wire, but each input data stream must be switched onto a different output wire. Furthermore -and this is the tricky constraint- no two data streams can pass through the same junction box following the switching operation. Finally, here's the problem. Show that for any specified pattern in which the input wires and output wires meet each other (each pair meeting exactly once), a valid switching of the data streams can always be found -one in which each input data stream is switched onto a different output, and no two of the resulting streams pass through the same junction box. Additionally, give an algorithm to find such a valid switching.

Solution

Observation 1 Every input wire would prefer its data stream to be switched as early as possible, to minimize the risk of running into another data stream that has not been switched, at a junction box. Similarly, every output wire wants its stream to be switched as late as possible (closest to terminus) to avoid running into another data stream at a junction box.

Initially every input wire has not been switched onto any output wire
while there is an input wire that has not been switched and it hasn't tried every output wire **do**
    Choose such an input wire i

Let o be the closest output wire to the source of i, to which i has not yet been switched to
if o is not yet switched to an input **then**
(i, o) ▷ i gets switched to o
else if o is already connected to input i' **then**
if i is closest to terminus than i' **then**
(i, o) ▷ i gets switched to o
i' gets switched out
else if i' is closest to terminus than i **then**
i is not switched onto o
end if
end if
end while
return the set S of switches

Observation 2 Once an output wire gets an input stream, it never gets switched out. It only gets switched onto inputs closer to terminus.
Claim Every input data stream is switched onto a different output.
Proof (By contradiction)
Suppose i is an input wire not switched onto any output wire.
→ There is an output wire o that has not been switched onto.
→ (By observation 2) i never tried to switch onto o
But i switched onto every output wire, since it ends up not been switched onto an output wire! A contradiction.
Claim No two resulting streams pass through the same junction box.
Proof (By contradiction)
Let's suppose there are two streams that pass through the same junction box.
→ There is an input wire i that got switched onto output wire o and was further from terminus than another wire i', connected to output o'.
This can only happen if one of the following is true:
Case 1 i' never switched onto o
→ o' is closest to source from i' pov than o.
→ i' doesn't pass through any junction box that outputs to o.
Case 2 o prefered i over i'
But that cannot happen, since i' is closer to terminus than i!

In either case i and i' don't pass through the same junction box. □

Chapter 2

Basics of Algorithm Analysis

2.1 Exercises

1. Suppose you have algorithms with the six running times listed below. (Assume these are the exact number of operations performed as a function of input size n.) Suppose you have a computer that can perform 10^{10} operations per second, and you need to compute a result in at most an hour of computation. For each of the algorithms, what is the largest input size n for which you would be able to get the result within an hour?

   (a) n^2
   (b) n^3
   (c) $100n^2$
   (d) $n \log n$
   (e) 2^n
   (f) 2^{2^n}

   Solution
   For each function $f(n)$, we need to find the maximum natural number n_0 so that: $f(n_0) \leq 3600 \times 10^{10}$.

   (a)

   $$\begin{aligned} n^2 &\leq 3600 \times 10^{10} \\ n &\leq \sqrt{3600 \times 10^{10}} \\ n &\leq 60 \times 10^5 \\ n &= 6 \times 10^6 \end{aligned}$$

   (b)

   $$\begin{aligned} n^3 &\leq 3600 \times 10^{10} \\ n &\leq \sqrt[3]{3600 \times 10^{10}} \\ n &= \lfloor 33019.27 \rfloor \\ n &= 33019 \end{aligned}$$

(c)

$$\begin{aligned} 100n^2 &\leq 3600 \times 10^{10} \\ n^2 &\leq 36 \times 10^{10} \\ n &= 6 \times 10^5 \end{aligned}$$

(d) We have $n \log n \leq 3600 \times 10^{10}$. To solve this inequality, we apply numerical methods. According to Newton-Raphson, $n \approx 1.3 \times 10^{12}$.

(e)

$$\begin{aligned} 2^n &\leq 3600 \times 10^{10} \\ n &\leq \log_2(3600 \times 10^{10}) \\ n &= \lfloor 45.03 \rfloor \\ n &= 45 \end{aligned}$$

(f)

$$\begin{aligned} 2^{2^n} &\leq 3600 \times 10^{10} \\ 2^n &\leq \log_2\left(3600 \times 10^{10}\right) \\ n &\leq log_2\left(45.03\right) \\ n &= \lfloor 5.49 \rfloor \\ n &= 5 \end{aligned}$$

2. Consider the following basic problem. You're given an array A consisting of n integers $A[1], A[2], ..., A[n]$. You'd like to output a two-dimensional n-by-n array B in which $B[i, j]$ (for $i < j$) contains the sum of array entries $A[i]$ through $A[j]$ - that is, the sum $A[i] + A[i+1] + ... + A[j]$. (The value of array entry $B[i, j]$ is left unspecified whenever $i \geq j$, so it doesn't matter what is output for these values.)
Here is a simple algorithm to solve this problem.

for $i = 1, 2, ..., n$ **do**
  for $j = i + 1, i + 2, ..., n$ **do**
    Add up array entries $A[i]$ through $A[j]$
    Store the result in $B[i, j]$
  end for
end for

(a) For some function f that you should choose, give a bound of the form $O(f(n))$ on the running time of this algorithm on an input of size n (*i.e.*, a bound on the number of operations performed by the algorithm).

(b) For this same function f, show that the running time of the algorithm on an input of size n is also $\Omega(f(n))$. (This shows an asymptotically tight bound of $\Theta(f(n))$ on the running time.)

(c) Although the algorithm you analyzed in parts (a) and (b) is the most natural way to solve the problem - after all, it just iterates through the relevant entries of the array B, filling in a value for each - it contains some highly unnecessary sources of inefficiency. Give a different algorithm to solve this problem, with an symptotically better running time. In other words, you should design an algorithm with running time $O(g(n))$, where $\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$.

Solution

(a) Claim. This algorithm has a running complexity of $O(n^3)$.
Proof. The first loop runs for n iterations. The second runs at most $n-1$ iterations, for n sufficiently large $n-1 \approx n$. Adding entries $A[i]$ through $A[j]$ takes $j-i+1$ steps, and at most n steps (for $i=1, j=n$). Storing entries takes constant time. Since this addition is repeated at worst n^2 times, this algorithm is $O(n^3)$.

(b) We have the lowest iterations when i is small in comparison to n, and j is close to n. Arbitrarily let's choose $i \leq \frac{n}{10}$ and $j \geq \frac{9n}{10}$. So the addition operation is repeated $\left(\frac{n}{10}\right)^2$ times. Each one takes at least $j-i+1 = \frac{9n}{10} - \frac{n}{10} + 1 = \frac{4n}{5} + 1$ steps (the fewest number of elements between the values of i and j). In total we have $\left(\frac{n}{10}\right)^2 \frac{4n}{5} = \frac{n^3}{125}$, which is just $\Omega(n^3)$.

(c) Consider the following algorithm:

Initialize B as an n x n array
$B[1][1] \leftarrow A[1]$
for $k = 2, 3, ..., n$ **do**
  $B[1][k] \leftarrow B[1][k-1] + A[k]$
end for
for $i = 2, 3, ..., n$ **do**
  for $j = i+1, i+2, ..., n$ **do**
    $B[i][j] \leftarrow B[i-1][j] - A[i-1]$
  end for
end for

Claim. This algorithm solves the problem. (By induction)

We need to prove that for any $1 \leq i < j \leq n$, $B[i][j] = A[i] + A[i+1] + ... + A[j]$. Given the nature of the algorithm, we use mathematical induction on the row index.

Proof. (By induction)
Base case.
$B[1][1]$ is assigned to be $A[1]$
For every $k \leq n$, the algorithm assigns $B[1][k] \leftarrow B[1][k-1] + A[k]$

By expansion we get

$$B[1][k] = A[1] + A[2] + ... + A[k-1] + A[k]$$

Thus the first row is assigned correctly.
Inductive Step.
Inductive Hypotesis. We assume that for some row $k < n$, every entry is correctly assigned, so that $B[k][j] = A[k] + A[k+1] + ... + A[j-1] + A[j]$ for all $k < j \leq n$.
We need to show that row $k+1$, the algorithm correctly assigns every entry.
The algorithm assigns $B[k+1][j] \leftarrow B[k][j] - A[k]$. (For every $k < j \leq n$)
If we substitute our inductive hypotesis, we get

$$B[k+1][j] = (A[k] + A[k+1] + ... + A[j-1] + A[j]) - A[k]$$
$$B[k+1][j] = A[k+1] + ... + A[j-1] + A[j],$$

which is the correct assignment for this entry.

□

Claim. This algorithm runs in $O(n^2)$.

Proof. The first loop runs for n iterations. The second loop is not associated with the first one; it runs for $n-1 \approx n$ iterations (for n sufficiently large). The third loop runs at most $n-2 \approx n$ iterations. The assignation to entry $B[i][j]$ is done in constant time. Since this assignation is repeated at worst n^2 times, this algorithm is $O(n^2)$.

3. You're doing some stress-testing on various models of glass jars to determine the height from which they can be dropped and still not break. The setup for this experiment, on a particular type of jar, is as follows. You have a ladder with n rungs, and you want to find the highest rung from which you can drop a copy of the jar and not have it break. We call this the *highest safe rung*.
It might be natural to try binary search: drop a jar from the middle rung, see if it breaks and then recursively try from rung $n/4$ or $3n/4$ depending on the outcome. But this has the drawback that you could break a lot of jars in finding the answer.
If your primary goal were to conserve jars, on the other hand, you could try the following strategy. Start by dropping a jar from the first rung, then the second rung, and so forth, climbing one higher each time until the jar breaks. In this way, you only need a single jar - at the moment it breaks, you have the correct answer - but you may have to drop it n times (rather than $\log n$ as in binary search solution).
So here is the trade-off: it seems you can perform fewer drops if you're willing to break more jars. To understand better how this trade-off works at a quantitative level, let's consider how to run this experiment given a fixed "budget" of $k \geq 1$ jars. In other words, you have to determine the correct answer - the highest safe rung - and can use at most k jars in doing so.

(a) Suppose you are given a budget of $k = 2$ jars. Describe a strategy for finding the highest safe rung that requires you to drop a jar at most $f(n)$ times, for some function $f(n)$ that grows slower than linearly. (In other words, it should be the case that $\lim_{n\to\infty} \frac{f(n)}{n} = 0$.)

(b) Now suppose you have a budget of $k > 2$ jars, for some given k. Describe a strategy for finding the highest safe rung using at most k jars. If $f_k(n)$ denotes the number of times you need you drop a jar according to your strategy, the functions $f_i, f_2, f_3, ...$ should have the property that each grows asymptotically slower than the previous one: $\lim_{n\to\infty} \frac{f_k(n)}{f_{k-1}(n)} = 0$ for each k.

Solution

(a) Consider the following algorithm:

$i \leftarrow \lceil \sqrt{n} \rceil$
for $k = 1, 2, ..., i$ **do**
  if $k = i$ **then**
    Throw the first jar from rung n
  else
    Throw the first jar from rung $k * i$
  end if
  if the jar breaks **then**
    Apply the algorithm for 1 jar, starting from rung $(k-1) * i + 1$
  end if
end for
return n

Claim. This algorithm finds the highest safe rung.

Proof (by contradiction). Let J be the rung returned by the algorithm and H be the actual highest safe rung. Let's assume $J \neq H$. Therefore, we have two cases.

Case 1. $J > H$
$\rightarrow J$ is unsafe.
But the algorithm returned J because the second jar was checked at position $J+1$ and the jar broke.
$\rightarrow$ The algorithm checked at position J and the jar didn't break.
$\rightarrow J$ is safe. A contradiction.

Case 2. $J < H$
$\rightarrow J + 1 \leq H$
$\rightarrow J + 1$ is safe.
Once again, the algorithm returned J because the second jar was checked at position $J + 1$ and the jar broke.

$\rightarrow J + 1$ is unsafe. A contradiction.

Since our assumption that $J \neq H$ ended up in a contradiction in both cases, that implies $J = H$. In other words, the algorithm finds the highest safe rung. $\square$

Claim. This algorithm has a running complexity of $O(\sqrt{n})$.

Proof. Since the first loop stops when $k = i$, then it runs at most for $\lceil \sqrt{n} \rceil$ iterations. Thus, $\sqrt{n} \leq k < \sqrt{n} + 1$. So we have an upper bound of $\sqrt{n} + 1$. Then if the first jar doesn't break at the last iteration, we are done. Otherwise, we now start dropping the second jar between $(k-1)\sqrt{n} + 1$ and $k\sqrt{n}$. This interval clearly has a size of $\sqrt{n}$, at worst. The algorithm for 1 jar runs at worst z iterations, being z the length of the interval. Therefore, in the worst case we check $(\sqrt{n} + 1) + (\sqrt{n} - 1) = 2\sqrt{n}$ rungs. Which gives us a running complexity of $O(\sqrt{n})$. $\square$

(b) Since we need an algorithm so that $\lim_{n\to\infty} \dfrac{f_k(n)}{f_{k-1}(n)} = 0$, intuitively we may think that for $k = 3$, $f_3(n) = 3n^{1/k}$.

Bibliography

[1] Jon Kleinberg and Éva Tardos. *Algorithm Design.* Addison Wesley, 2006.